Diabetic Retinopathy Detection from Retinal Fundus Images using a Convolutional Neural Network (CNN) with MobileNetV2 Feature Extraction

(Diabetic Retinopathy Severity Classification using Deep Learning)

Problem Statement:

Diabetic Retinopathy (DR) is a diabetes-related complication that damages the tiny blood vessels of the retina and is one of the leading causes of preventable vision loss among working-age adults worldwide. In its early stages it produces few or no symptoms, and by the time the patient notices blurred vision, considerable and often irreversible damage has already occurred. Manual grading of retinal fundus images by an ophthalmologist is accurate but time-consuming, subjective, and difficult to scale to the large diabetic population, particularly in regions with a shortage of eye-care specialists.

There is, therefore, a need for an automated, fast, and accessible system that can analyze a retinal fundus image and grade it into one of five severity levels — No DR, Mild, Moderate, Severe, or Proliferative DR — with a measurable confidence score, so that patients at risk can be referred early for specialist review. This project designs and implements a web-based Diabetic Retinopathy Detection system that performs image preprocessing (Resize, Gaussian blur, CLAHE), deep feature extraction using a pre-trained MobileNetV2 network, and classification using a Convolutional Neural Network (CNN) built with TensorFlow, and evaluates the trained model using standard classification metrics (Accuracy, Precision, Recall, F1-Score, Confusion Matrix).

Objectives:

1. To load retinal fundus images and preprocess them (resizing, noise reduction and contrast enhancement) for diabetic retinopathy analysis.
2. To extract meaningful deep features from each image using a pre-trained MobileNetV2 network and visualize the resulting feature maps.
3. To design, train, and save a CNN-based classifier that grades DR into five severity classes (No DR, Mild, Moderate, Severe, Proliferative).
4. To build a Flask-based backend and a web front-end that lets a user upload a fundus image and step through the complete ML pipeline.
5. To predict the DR severity of an unseen image and display the class-wise probability (confidence) for all five classes.
6. To evaluate the trained model and report Accuracy, Precision, Recall, F1-score and the Confusion Matrix.

Software / Hardware Required:

• Operating System: Windows / Linux / macOS

• IDE: Visual Studio Code

• Language: Python 3.11

• Backend Framework: Flask (REST API)

• Deep Learning: TensorFlow / Keras (MobileNetV2, CNN)

• Image Processing: OpenCV (cv2), Pillow (PIL), NumPy

• Frontend: HTML5, CSS3, JavaScript (Chart.js for training curves)

• Dataset: Retinal fundus image dataset with five DR severity grades (Class 0 – Class 4)

• Version Control: Git

Project Directory Structure:

The project ("Diabetic_Retinopathy") is organized as a Flask application (app.py) with a separate machine-learning module (dr_model.py), a static front-end folder, and an uploads folder, as shown below.

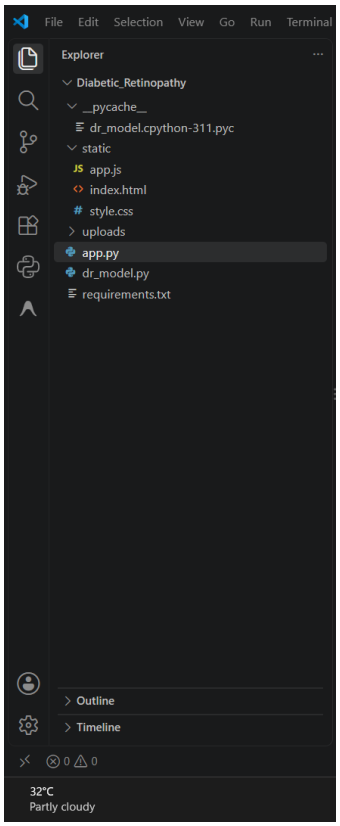


Fig 1: Project directory structure of Diabetic_Retinopathy in Visual Studio Code

Procedure:

1. Launch the Flask application (python app.py) and open the web front-end at http://localhost:5000 in the browser.

2. Load / upload a retinal fundus image (JPG, PNG, BMP or TIFF) through the drag-and-drop interface (Step 1).

3. Preprocess the image — resize to 224×224, apply Gaussian blur (3×3) for noise reduction and CLAHE (clip limit 2.0, tile grid 8×8) for contrast enhancement, and compare the before / after images (Step 2).

4. Pass the preprocessed image through the MobileNetV2 feature extractor to obtain a 56×56×144 feature output and display representative feature maps (Step 3).

5. Train the CNN classifier (dr_model.py) for 5 epochs and monitor the epoch-wise training loss, validation loss, training accuracy and validation accuracy along with the accuracy and loss curves (Step 4).

6. Feed the preprocessed image to the trained model to predict the DR severity class (No DR / Mild / Moderate / Severe / Proliferative) with class-wise probabilities and a confidence percentage (Step 5).

7. Evaluate the model and compute Accuracy, Precision, Recall, F1-score and the Confusion Matrix (Step 6).

Implementation Code:

Representative implementation of the key modules used in the project, based on the pipeline shown in the application (adapt variable/file names to match your exact source files).

1. Image Preprocessing (dr_model.py)

```
import cv2
import numpy as np
def preprocess_image(image_path, target_size=(224, 224)):
    img = cv2.imread(image_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)        # RGB conversion
    img = cv2.resize(img, target_size)                # Resize to 224x224
    img = cv2.GaussianBlur(img, (3, 3), 0)            # Noise reduction
    lab = cv2.cvtColor(img, cv2.COLOR_RGB2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    l = clahe.apply(l)                                # CLAHE contrast enhancement
    img = cv2.cvtColor(cv2.merge((l, a, b)), cv2.COLOR_LAB2RGB)
    img = img.astype(np.float32) / 255.0              # Normalize to [0, 1]
    return img
```

2. Feature Extraction using MobileNetV2 (dr_model.py)

```
import numpy as np
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Model
base = MobileNetV2(weights='imagenet', include_top=False,
                   input_shape=(224, 224, 3))
# Early layer output -> 56 x 56 x 144 feature maps
feature_model = Model(inputs=base.input,
                      outputs=base.get_layer('block_2_expand_relu').output)
def extract_feature_maps(img):
    x = np.expand_dims(img, axis=0)                  # (1, 224, 224, 3)
    fmap = feature_model.predict(x)[0]               # (56, 56, 144)
    return fmap                                  # 9 maps are visualized in the UI
```

3. CNN Model Building and Training (dr_model.py)

```
from tensorflow.keras import layers, models
CLASSES = ['No DR', 'Mild', 'Moderate', 'Severe', 'Proliferative']
base.trainable = False                           # use pre-trained MobileNetV2
model = models.Sequential([
    base,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(5, activation='softmax')            # 5 DR severity classes
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
history = model.fit(X_train, y_train,
                    validation_data=(X_val, y_val),
                    epochs=5, batch_size=16)
model.save('dr_model.h5')
```

4. Prediction Route (app.py)

```
from flask import Flask, request, jsonify
from tensorflow.keras.models import load_model
from dr_model import preprocess_image, CLASSES
import numpy as np, os
app = Flask(__name__, static_folder='static', static_url_path='')
```

```
model = load_model('dr_model.h5')
@app.route('/')
def index():
    return app.send_static_file('index.html')
@app.route('/predict', methods=['POST'])
def predict():
    file = request.files['image']
    path = os.path.join('uploads', file.filename)
    file.save(path)
    img = preprocess_image(path)
    proba = model.predict(np.expand_dims(img, axis=0))[0]
    idx = int(np.argmax(proba))
    return jsonify({
        'prediction': CLASSES[idx],
        'confidence': round(float(proba[idx]) * 100, 2),
        'probabilities': {c: round(float(p) * 100, 2)
                          for c, p in zip(CLASSES, proba)}
    })
if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

5. Model Evaluation (dr_model.py)

```
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, confusion_matrix)
y_pred = np.argmax(model.predict(X_val), axis=1)
print('Accuracy :', accuracy_score(y_val, y_pred))
print('Precision:', precision_score(y_val, y_pred, average='weighted', zero_division=0))
print('Recall   :', recall_score(y_val, y_pred, average='weighted', zero_division=0))
print('F1-Score :', f1_score(y_val, y_pred, average='weighted', zero_division=0))
print(confusion_matrix(y_val, y_pred))
```

Output (Step-by-Step Screenshots):

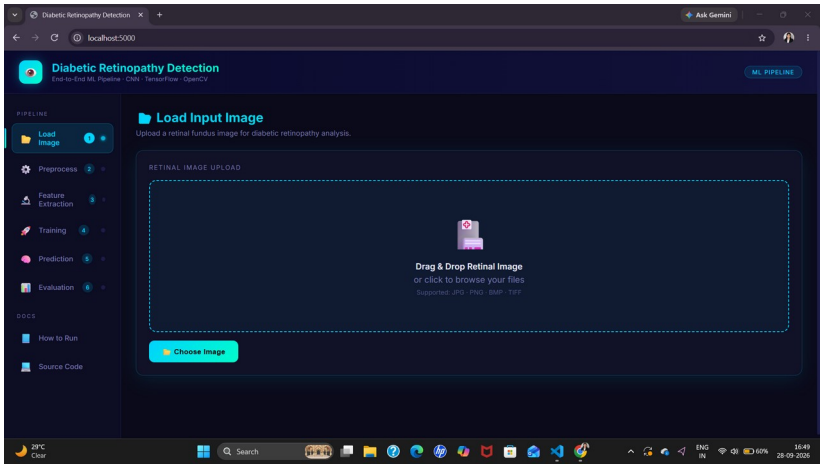


Fig 2: Step 1 — Load Input Image: drag-and-drop interface to upload a retinal fundus image

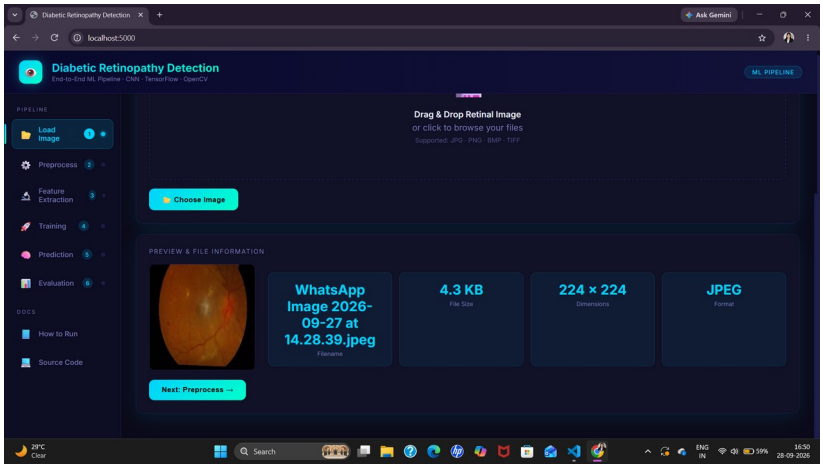


Fig 3: Step 1 — Input image preview with file information (name, size, 224×224 dimensions, format)

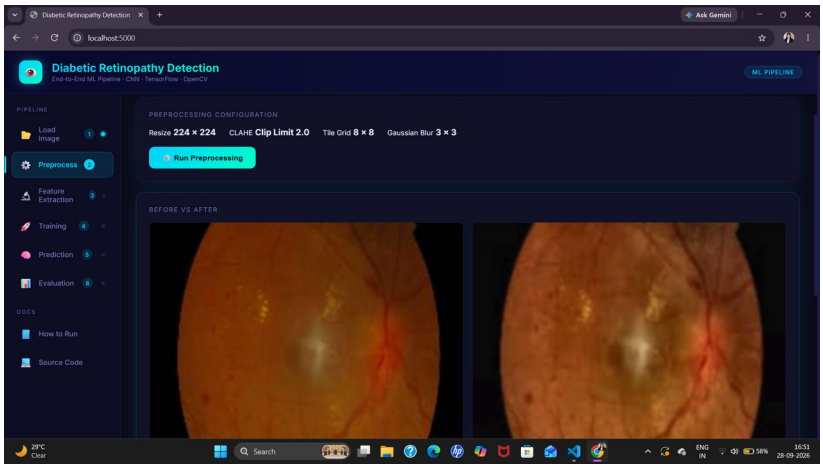


Fig 4: Step 2 — Image preprocessing: original image vs. preprocessed image (Resize 224×224, CLAHE clip limit 2.0, tile grid 8×8, Gaussian blur 3×3)

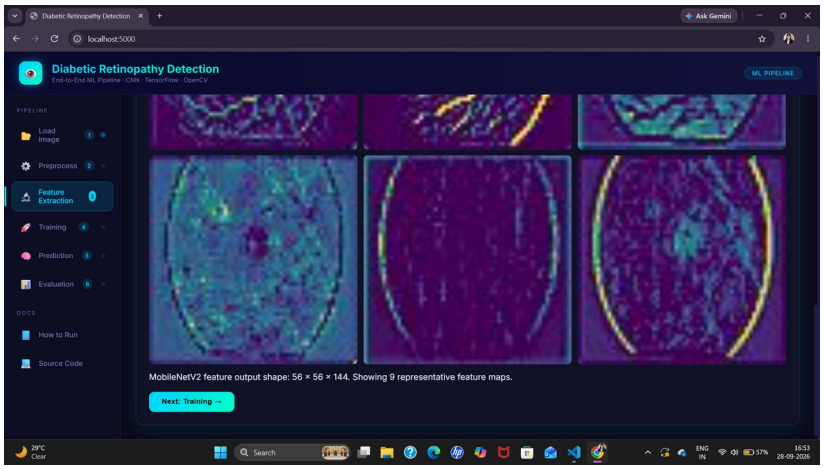


Fig 5: Step 3 — Feature extraction: MobileNetV2 feature output (56×56×144) with 9 representative feature maps



Fig 6: Step 4 — Model training: epoch-wise training/validation loss and accuracy with accuracy and loss curves (5 epochs)

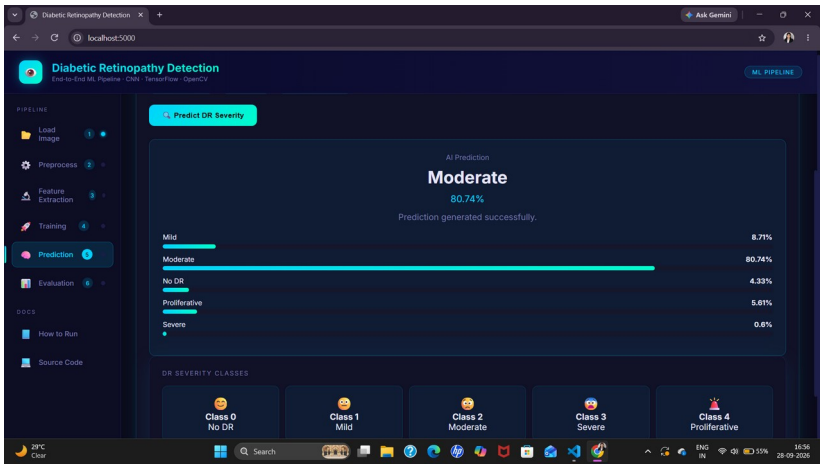


Fig 7: Step 5 — Prediction result: Moderate DR predicted with 80.74% confidence and class-wise probabilities

[Insert the Step 6 — Evaluation screenshot here (Accuracy, Precision, Recall, F1-Score and Confusion Matrix) and caption it: "Fig 8: Step 6 — Performance evaluation metrics and confusion matrix".]

Result:

A web-based Diabetic Retinopathy Detection system was successfully designed and implemented using a Flask backend, OpenCV-based preprocessing (Resize, Gaussian blur, CLAHE), MobileNetV2 feature extraction and a TensorFlow CNN classifier. A retinal fundus image is loaded, preprocessed, passed through the feature extractor, and graded into one of five severity classes (No DR, Mild, Moderate, Severe, Proliferative) with class-wise probabilities. For the sample fundus image tested, the system predicted Moderate DR with a confidence of 80.74%.

During the 5-epoch training run, the training loss decreased from 1.8522 to 1.5733 and the training accuracy improved from 19.50% to 28.50%, while the validation accuracy remained at about 16.00% and the validation loss rose to 2.5125. This indicates that the model is still under-trained on the small dataset used; training on a larger labelled dataset for more epochs (with data augmentation and fine-tuning of the MobileNetV2 layers) is expected to improve the generalization and the evaluation metrics.